

# APACHE SPARK FUNDAMENTALS

RDDs · DataFrames · Spark SQL · Reading & Writing Data (CSV, Parquet, Delta Lake)

---

*Corporate Training Module*

Hands-on Session Guide with PySpark Examples

Prepared for Data Engineering & Analytics Training

# Table of Contents

---

## Session Overview3

### Module 1: Introduction to Apache Spark4

- 1.1 What is Apache Spark?4
- 1.2 Spark vs. Traditional Hadoop MapReduce4
- 1.3 Spark Architecture4
- 1.4 The Spark Ecosystem5

### Module 2: RDDs (Resilient Distributed Datasets)6

- 2.1 What is an RDD?
- 2.2 Creating RDDs6
- 2.3 Transformations vs. Actions6
- 2.4 Lazy Evaluation and the DAG7
- 2.5 Persistence: cache() and persist()8
- 2.6 When Should You Use RDDs?9

### Module 3: DataFrames10

- 3.1 What is a DataFrame?10
- 3.2 Creating DataFrames10
- 3.3 Schemas: Inferred vs. Explicit10
- 3.4 Core DataFrame Operations11
- 3.5 How Catalyst and Tungsten Speed Things Up12

### Module 4: Spark SQL13

- 4.1 Running SQL Directly on DataFrames13
- 4.2 Types of Views13
- 4.3 The Spark Catalog13
- 4.4 DataFrame API vs. SQL: Side by Side14
- 4.5 User-Defined Functions (UDFs)14

### Module 5: Reading & Writing Data — CSV, Parquet, Delta15

- 5.1 The DataFrameReader and DataFrameWriter15
- 5.2 CSV15
- 5.3 Parquet15
- 5.4 Delta Lake16
- 5.5 CSV vs. Parquet vs. Delta — Choosing a Format17
- 5.6 Write Modes17
- 5.7 Partitioning on Write17

### Module 6: Hands-On Lab — Retail Sales Pipeline19

### Module 7: Summary & Key Takeaways21

# Session Overview

This module introduces Apache Spark, the industry-standard engine for large-scale data processing, and builds working fluency across its three core programming abstractions — RDDs, DataFrames, and Spark SQL — before applying that fluency to real file formats (CSV, Parquet, and Delta Lake). The session is designed to run as a hands-on classroom module, with runnable PySpark code, comparison tables, and a guided lab exercise.

## Learning Objectives

By the end of this session, participants will be able to:

1. Explain Spark's architecture (Driver, Executors, Cluster Manager) and how it distributes work across a cluster.
2. Create and transform RDDs, and explain lazy evaluation and the DAG execution model.
3. Create, inspect, and manipulate DataFrames using the PySpark DataFrame API.
4. Write and execute Spark SQL queries against DataFrames using temporary views.
5. Read and write data in CSV, Parquet, and Delta Lake formats, and choose the right format for a given use case.
6. Apply partitioning, caching, and write-mode strategies for efficient pipelines.

## Prerequisites

- Working knowledge of Python (functions, lists, dictionaries).
- Basic SQL (SELECT, WHERE, GROUP BY, JOIN).
- A Spark environment — Databricks Community Edition, local PySpark install, or any managed Spark cluster.

## Session Agenda

| # | Topic                                        | Duration |
|---|----------------------------------------------|----------|
| 1 | Introduction to Apache Spark & Architecture  | 30 min   |
| 2 | RDDs — Concepts, Transformations, Actions    | 45 min   |
| 3 | DataFrames — Creation & Operations           | 45 min   |
| 4 | Spark SQL — Views, Queries, Catalog          | 30 min   |
| 5 | Reading & Writing Data — CSV, Parquet, Delta | 45 min   |
| 6 | Hands-on Lab & Wrap-up                       | 45 min   |

# Module 1: Introduction to Apache Spark

## 1.1 What is Apache Spark?

Apache Spark is an open-source, distributed data processing engine designed for speed, ease of use, and sophisticated analytics on datasets ranging from gigabytes to petabytes. Instead of processing data on a single machine, Spark splits data and computation across many machines (a cluster), running tasks in parallel and combining the results.

Spark was originally built to overcome the limitations of Hadoop MapReduce, which writes intermediate results to disk after every step. Spark instead keeps data in memory across operations wherever possible, which is why it is commonly 10 to 100 times faster than MapReduce for iterative workloads such as machine learning training or interactive queries.

### Why Organizations Use Spark

- **Speed:** in-memory computation and an optimized execution engine (Catalyst + Tungsten) reduce runtime dramatically compared to disk-based engines.
- **Unified engine:** one engine handles batch processing, SQL analytics, streaming, machine learning (MLlib), and graph processing (GraphX).
- **Language flexibility:** APIs available in Python (PySpark), Scala, Java, R, and SQL — teams can work in the language they already know.
- **Scalability:** the same code that runs on a laptop with a few files can run unchanged on a 1,000-node cluster processing petabytes.
- **Rich ecosystem:** native connectors for cloud storage (S3, ADLS, GCS), data warehouses, Kafka, and lakehouse formats like Delta Lake and Iceberg.

## 1.2 Spark vs. Traditional Hadoop MapReduce

| Aspect           | Hadoop MapReduce                                                 | Apache Spark                                                     |
|------------------|------------------------------------------------------------------|------------------------------------------------------------------|
| Processing model | Disk-based; writes intermediate results to HDFS after every step | In-memory; keeps intermediate data in RAM across stages          |
| Speed            | Slower — disk I/O bound                                          | 10–100x faster for iterative jobs                                |
| Ease of use      | Verbose Java API, only batch processing                          | Concise APIs (Python/Scala/SQL); batch + streaming + ML          |
| Fault tolerance  | Replicates data on disk (HDFS)                                   | Recomputes lost data using lineage (DAG) — no replication needed |
| Use cases        | Simple, one-pass batch ETL                                       | ETL, iterative ML, streaming, interactive SQL analytics          |

## 1.3 Spark Architecture

A Spark application runs as a set of coordinated processes across a cluster. Understanding these pieces is essential to reasoning about performance and failures.

- **Driver Program:** the process that runs your `main()` function / notebook code. It creates the `SparkSession`, builds the logical execution plan (the DAG), and schedules work — but does not process data itself.
- **SparkSession:** the single entry point to all Spark functionality (DataFrame API, SQL, configuration). In Databricks notebooks, a `SparkSession` is pre-created as the variable `spark`.
- **Cluster Manager:** allocates resources (CPU, memory) across the cluster. Options include Spark's Standalone manager, YARN, Kubernetes, or a managed service like Databricks' cluster manager.
- **Executors:** worker processes launched on cluster nodes that actually run tasks and store data for the application, in memory or on disk. Each executor runs many tasks in parallel across its allocated CPU cores.
- **Tasks:** the smallest unit of work — a single transformation applied to one partition of data. Many tasks execute in parallel across executors.

#### How it fits together

When you run a Spark job, the Driver asks the Cluster Manager for resources, launches Executors on worker nodes, splits your data into partitions, and sends tasks to Executors to process those partitions in parallel. Results are sent back to the Driver or written directly to storage.

## 1.4 The Spark Ecosystem

| Component            | Purpose                                                                                  |
|----------------------|------------------------------------------------------------------------------------------|
| Spark Core           | Base engine: task scheduling, memory management, fault recovery, RDD API                 |
| Spark SQL            | Structured data processing via DataFrames and SQL queries                                |
| Structured Streaming | Near real-time processing of streaming data using the same DataFrame API                 |
| MLlib                | Distributed machine learning library (classification, regression, clustering, pipelines) |
| GraphX / GraphFrames | Graph-parallel computation (e.g., network/relationship analysis)                         |

This session focuses on Spark Core (RDDs) and Spark SQL (DataFrames and SQL queries) — the foundation on which everything else in the ecosystem is built.

## Module 2: RDDs (Resilient Distributed Datasets)

### 2.1 What is an RDD?

An RDD (Resilient Distributed Dataset) is Spark's original, low-level data abstraction: an immutable, partitioned collection of objects that can be processed in parallel across a cluster. RDDs are the foundation that DataFrames and Spark SQL are built on top of — every DataFrame operation is ultimately compiled down to RDD operations.

The name describes its core properties:

- **Resilient:** if a partition of data is lost (e.g., a node fails), Spark can recompute it using lineage information — the recorded sequence of transformations that produced it — instead of relying on data replication.
- **Distributed:** data is split into partitions spread across the executors in the cluster, enabling parallel processing.
- **Dataset:** a collection of data — records, rows, or arbitrary Python/Scala objects.
- **Immutable:** an RDD cannot be changed after creation. Every transformation produces a brand-new RDD rather than modifying the original.

### 2.2 Creating RDDs

There are two common ways to create an RDD:

7. Parallelizing an existing collection in your driver program.
8. Loading data from external storage (local file system, HDFS, S3, ADLS).

```
# 1. Create an RDD from a Python list
cities = ['Ahmedabad', 'Mumbai', 'Bengaluru', 'Pune', 'Jaipur']
cities_rdd = spark.sparkContext.parallelize(cities)

# 2. Create an RDD by reading a text file (one element per line)
orders_rdd = spark.sparkContext.textFile('/data/retail/orders.txt')

print(cities_rdd.getNumPartitions()) # number of partitions
```

### 2.3 Transformations vs. Actions

RDD operations fall into two categories. This distinction is one of the most important concepts in all of Spark, since it also applies to DataFrames.

- **Transformations:** produce a new RDD from an existing one (e.g., map, filter). Transformations are lazy — Spark does not execute them immediately; it just records what should happen.
- **Actions:** trigger actual computation and either return a result to the driver or write data to storage (e.g., collect, count, saveAsTextFile). Only when an action is called does Spark execute the chain of transformations.

#### Common Transformations

| Transformation    | Description                                | Example                                     |
|-------------------|--------------------------------------------|---------------------------------------------|
| map(func)         | Applies func to every element, 1-to-1      | rdd.map(lambda x: x.upper())                |
| filter(func)      | Keeps elements where func returns True     | rdd.filter(lambda x: x.startswith('P'))     |
| flatMap(func)     | Like map, but flattens results (1-to-many) | rdd.flatMap(lambda line: line.split(' '))   |
| distinct()        | Removes duplicate elements                 | rdd.distinct()                              |
| union(other)      | Combines two RDDs                          | rdd1.union(rdd2)                            |
| groupByKey()      | Groups values by key (for key-value RDDs)  | pairs.groupByKey()                          |
| reduceByKey(func) | Combines values per key using func         | pairs.reduceByKey(lambda a,b: a+b)          |
| sortBy(func)      | Sorts elements by a key function           | rdd.sortBy(lambda x: x[1], ascending=False) |

### Common Actions

| Action               | Description                                  | Example                           |
|----------------------|----------------------------------------------|-----------------------------------|
| collect()            | Returns all elements to the driver as a list | rdd.collect()                     |
| count()              | Returns the number of elements               | rdd.count()                       |
| first()              | Returns the first element                    | rdd.first()                       |
| take(n)              | Returns the first n elements                 | rdd.take(5)                       |
| reduce(func)         | Aggregates all elements using func           | rdd.reduce(lambda a,b: a+b)       |
| saveAsTextFile(path) | Writes the RDD to storage as text files      | rdd.saveAsTextFile('/out/cities') |

## 2.4 Lazy Evaluation and the DAG

When you call a transformation, Spark does not touch your data — it simply adds a step to a logical plan called a Directed Acyclic Graph (DAG). Only when an action is called does Spark's scheduler analyze the DAG, break it into stages and tasks, and send work to the executors.

### Why laziness matters

Because Spark sees the full chain of transformations before running anything, it can optimize the plan — combining steps, reordering filters to run earlier, and avoiding unnecessary computation. This is very different from row-by-row imperative code, where each line executes immediately.

## Worked Example: Word Count on Indian City Names

```
text_rdd = spark.sparkContext.parallelize([
    'Ahmedabad Gujarat India',
    'Jaipur Rajasthan India',
    'Mumbai Maharashtra India'
])

word_counts = (text_rdd
    .flatMap(lambda line: line.split(' ')) # transformation: split into words
    .map(lambda word: (word, 1))           # transformation: pair each word with 1
    .reduceByKey(lambda a, b: a + b))      # transformation: sum counts per word

# Nothing has executed yet -- all lazy so far.
result = word_counts.collect() # ACTION -- triggers execution
print(result)
# [('Ahmedabad', 1), ('Gujarat', 1), ('India', 3), ('Jaipur', 1), ...]
```

## 2.5 Persistence: cache() and persist()

Because RDDs are recomputed from lineage whenever needed, an RDD used multiple times will be recomputed each time by default — which wastes time. Calling `.cache()` or `.persist()` tells Spark to keep the RDD's data in memory (or memory + disk) after the first time it is computed, so subsequent actions reuse it instead of recomputing.

| Storage Level   | Description                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------|
| MEMORY_ONLY     | Default for <code>cache()</code> . Stores as deserialized Java/Python objects in RAM; recomputes partitions that don't fit. |
| MEMORY_AND_DISK | Spills to disk if an RDD doesn't fit fully in memory.                                                                       |
| DISK_ONLY       | Stores partitions only on disk.                                                                                             |
| MEMORY_ONLY_SER | Stores serialized (more compact, more CPU to read).                                                                         |

```
sales_rdd = spark.sparkContext.textFile('/data/retail/sales.csv')
sales_rdd.persist() # or sales_rdd.cache()

total = sales_rdd.count() # first action -- computes and caches
sample = sales_rdd.take(10) # second action -- reuses cached data, much faster
```

## 2.6 When Should You Use RDDs?

In modern Spark (2.x and later), the DataFrame API and Spark SQL are recommended for almost all workloads, because they benefit from the Catalyst query optimizer and Tungsten's memory management — RDDs do not get these automatic optimizations. RDDs are still useful for:

- Working with unstructured data (raw text, custom binary formats) that doesn't fit a tabular schema.
- Fine-grained control over partitioning and physical execution.



- Legacy code or libraries that expose only the RDD API.

**Practical guidance**

Teach RDDs to build a strong mental model of Spark's execution engine (lazy evaluation, lineage, partitions) — then default to DataFrames and Spark SQL for actual day-to-day data engineering work.

## Module 3: DataFrames

### 3.1 What is a DataFrame?

A DataFrame is a distributed collection of data organized into named columns — conceptually similar to a table in a relational database, a pandas DataFrame, or an Excel sheet, but distributed across a cluster. DataFrames are built on top of RDDs internally, but add a schema (column names and types) and are optimized by Spark's Catalyst query optimizer.

| RDD                                       | DataFrame                                  |
|-------------------------------------------|--------------------------------------------|
| No schema — just a collection of objects  | Has a schema: named, typed columns         |
| No automatic query optimization           | Optimized automatically by Catalyst        |
| Functional API only (map, filter, reduce) | Functional API + SQL queries               |
| Best for unstructured / custom data       | Best for structured / semi-structured data |

### 3.2 Creating DataFrames

```
# 1. From a list of tuples with an explicit schema
from pyspark.sql import Row
data = [('Anita Sharma', 'Ahmedabad', 54000),
        ('Rohan Mehta', 'Mumbai', 61000),
        ('Kavya Iyer', 'Bengaluru', 58500)]
df = spark.createDataFrame(data, ['employee_name', 'city', 'salary'])

# 2. From a CSV file (see Module 5 for full read options)
df_csv = spark.read.option('header', True).option('inferSchema', True) \
    .csv('/data/retail/employees.csv')

# 3. From an existing RDD
rdd = spark.sparkContext.parallelize(data)
df_from_rdd = rdd.toDF(['employee_name', 'city', 'salary'])

df.printSchema()
df.show(truncate=False)
```

### 3.3 Schemas: Inferred vs. Explicit

Spark can infer a schema by scanning the data (convenient, but slower and occasionally wrong on edge cases), or you can define it explicitly using StructType — the recommended approach for production pipelines, since it is faster and guarantees consistent types.

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType

schema = StructType([
    StructField('employee_name', StringType(), nullable=False),
```

```
StructField('city', StringType(), nullable=True),
StructField('salary', DoubleType(), nullable=True),
])

df = spark.read.schema(schema).option('header',
True).csv('/data/retail/employees.csv')
```

### 3.4 Core DataFrame Operations

| Operation                     | Purpose                  | Example                                 |
|-------------------------------|--------------------------|-----------------------------------------|
| select()                      | Choose specific columns  | df.select('employee_name', 'city')      |
| filter() / where()            | Filter rows by condition | df.filter(df.salary > 55000)            |
| withColumn()                  | Add or replace a column  | df.withColumn('bonus', df.salary * 0.1) |
| drop()                        | Remove a column          | df.drop('bonus')                        |
| groupBy().agg()               | Aggregate by group       | df.groupBy('city').agg(avg('salary'))   |
| orderBy() / sort()            | Sort rows                | df.orderBy(df.salary.desc())            |
| join()                        | Combine two DataFrames   | df1.join(df2, 'city', 'inner')          |
| distinct() / dropDuplicates() | Remove duplicate rows    | df.dropDuplicates(['employee_name'])    |
| na.fill() / na.drop()         | Handle missing values    | df.na.fill({'salary': 0})               |

#### Worked Example: Retail Sales DataFrame (Indian Cities)

```
from pyspark.sql import functions as F

sales_data = [
    ('Ahmedabad', 'Electronics', 125000),
    ('Ahmedabad', 'Groceries', 48000),
    ('Jaipur', 'Electronics', 98000),
    ('Pune', 'Groceries', 61000),
    ('Pune', 'Electronics', 143500),
]
sales_df = spark.createDataFrame(sales_data, ['city', 'category', 'revenue'])

# Total revenue per city, sorted highest first
city_totals = (sales_df
    .groupBy('city')
    .agg(F.sum('revenue').alias('total_revenue'))
    .orderBy(F.col('total_revenue').desc()))

city_totals.show()
# +-----+-----+
# | city|total_revenue|
# +-----+-----+
# | Pune|      204500|
```

```
# |Ahmedabad|      173000|  
# |   Jaipur|      98000|  
# +-----+-----+
```

### 3.5 How Catalyst and Tungsten Speed Things Up

Every DataFrame operation you write is translated into a logical plan, which Spark's Catalyst optimizer rewrites into an efficient physical plan — for example, pushing filters down to the data source, choosing efficient join strategies, and eliminating unused columns. Tungsten then manages memory and generates optimized bytecode for the physical plan. This is the core reason DataFrames typically outperform hand-written RDD code doing the same logical work.

**tip**

Show learners `df.explain()` on a chained DataFrame query so they can see the logical and physical plans Catalyst produces — it demystifies 'the optimizer' as a concrete, inspectable thing.

## Module 4: Spark SQL

### 4.1 Running SQL Directly on DataFrames

Spark SQL lets you query DataFrames using standard SQL syntax, which is often faster to write for people coming from a database background, and it produces the exact same optimized execution plan as the equivalent DataFrame API code — SQL and DataFrame operations are two syntaxes for the same underlying engine.

To run SQL against a DataFrame, first register it as a temporary view:

```
sales_df.createOrReplaceTempView('sales')

result = spark.sql('''
    SELECT city, SUM(revenue) AS total_revenue
    FROM sales
    GROUP BY city
    ORDER BY total_revenue DESC
''')
result.show()
```

### 4.2 Types of Views

| View Type                          | Scope                                                 | Method                                |
|------------------------------------|-------------------------------------------------------|---------------------------------------|
| Temporary View                     | Current SparkSession only                             | createOrReplaceTempView('name')       |
| Global Temporary View              | All sessions in the same Spark application            | createOrReplaceGlobalTempView('name') |
| Permanent Table (managed/external) | Persisted in the metastore, survives cluster restarts | df.write.saveAsTable('db.table_name') |

Global temporary views must be queried with the `global_temp` prefix, e.g. `SELECT * FROM global_temp.sales`.

### 4.3 The Spark Catalog

The Catalog is Spark's interface for exploring metadata: databases, tables, views, and columns currently known to your SparkSession.

```
spark.catalog.listDatabases()
spark.catalog.listTables()
spark.catalog.listColumns('sales')
```

### 4.4 DataFrame API vs. SQL: Side by Side

| Goal        | DataFrame API                                         | Spark SQL                                                       |
|-------------|-------------------------------------------------------|-----------------------------------------------------------------|
| Filter rows | <code>df.filter(df.revenue &gt; 100000)</code>        | <code>SELECT * FROM sales WHERE revenue &gt; 100000</code>      |
| Aggregate   | <code>df.groupBy('city').agg(F.avg('revenue'))</code> | <code>SELECT city, AVG(revenue) FROM sales GROUP BY city</code> |
| Join        | <code>df1.join(df2, 'city')</code>                    | <code>SELECT * FROM t1 JOIN t2 ON t1.city = t2.city</code>      |
| Sort        | <code>df.orderBy(F.col('revenue').desc())</code>      | <code>SELECT * FROM sales ORDER BY revenue DESC</code>          |

## 4.5 User-Defined Functions (UDFs)

When built-in functions cannot express your logic, you can register a Python function as a UDF for use in both the DataFrame API and SQL. UDFs are convenient but bypass some Catalyst optimizations, so built-in functions should always be preferred when available.

```
from pyspark.sql.types import StringType

def tag_revenue(revenue):
    return 'High' if revenue > 100000 else 'Standard'

tag_udf = spark.udf.register('tag_revenue', tag_revenue, StringType())

spark.sql('''
    SELECT city, revenue, tag_revenue(revenue) AS revenue_tier
    FROM sales
''').show()
```

## Module 5: Reading & Writing Data — CSV, Parquet, Delta

### 5.1 The DataFrameReader and DataFrameWriter

All file I/O in Spark SQL goes through two consistent interfaces: `spark.read` (the `DataFrameReader`) for reading data into a `DataFrame`, and `df.write` (the `DataFrameWriter`) for writing a `DataFrame` out to storage. The format changes; the pattern does not:

```
df = spark.read.format('<format>').option(...).load('<path>')
df.write.format('<format>').mode('<mode>').save('<path>')
```

### 5.2 CSV

CSV is the most common interchange format but the least efficient for large-scale analytics — it is row-based, uncompressed by default, has no embedded schema, and every read requires either a full scan to infer types or a manually supplied schema.

```
# Reading CSV
df = (spark.read
      .option('header', True)           # first row is column names
      .option('inferSchema', True)      # scan data to guess types (slower)
      .option('delimiter', ',')        # field separator
      .option('nullValue', 'NA')       # treat 'NA' as null
      .csv('/data/retail/sales.csv'))

# Writing CSV
df.write.mode('overwrite').option('header', True).csv('/data/output/sales_csv')
```

#### Watch out

`inferSchema` triggers an extra pass over the data just to guess types, which can be slow on large files. In production pipelines, prefer supplying an explicit `StructType` schema instead.

### 5.3 Parquet

Parquet is a columnar, compressed, binary file format built for analytics. Because it stores data column-by-column rather than row-by-row, Spark can read only the columns a query actually needs (column pruning) and skip irrelevant row groups using embedded statistics (predicate pushdown) — both of which dramatically reduce I/O compared to CSV.

- **Schema is embedded:** no need for `inferSchema` or a manually supplied schema — Parquet files carry their own column names and types.
- **Compression:** typically 3–5x smaller on disk than the equivalent CSV, reducing both storage cost and read time.
- **Splittable:** large Parquet files can be processed in parallel across many tasks.

```
# Reading Parquet -- no schema options needed, it's embedded
df = spark.read.parquet('/data/retail/sales.parquet')

# Writing Parquet, partitioned by city for faster filtered reads
df.write.mode('overwrite').partitionBy('city').parquet('/data/output/sales_parquet')
```

## 5.4 Delta Lake

Delta Lake is an open-source storage layer that sits on top of Parquet files, adding a transaction log (the `_delta_log` directory) that brings database-style reliability to a data lake. It is the default table format in Databricks and is central to the 'lakehouse' architecture — combining the low cost and flexibility of a data lake with the reliability of a data warehouse.

### What Delta Adds on Top of Parquet

- **ACID transactions:** writes either fully succeed or fully fail — concurrent readers never see a partially-written table.
- **Time travel:** query a previous version of a table using `VERSION AS OF` or `TIMESTAMP AS OF`, which is invaluable for audits and recovering from bad writes.
- **Schema enforcement & evolution:** Delta rejects writes that don't match the table schema by default, preventing silent data corruption, while still allowing controlled schema changes when needed.
- **UPSERT / MERGE support:** Delta supports `MERGE INTO` for combined insert-update-delete logic, which plain Parquet cannot do atomically.
- **Unified batch & streaming:** the same Delta table can be a sink for both batch jobs and Structured Streaming jobs.

```
# Reading a Delta table
df = spark.read.format('delta').load('/data/retail/sales_delta')

# Writing a Delta table
df.write.format('delta').mode('overwrite').save('/data/retail/sales_delta')

# Time travel -- query the table as it existed at an earlier version
df_v0 = spark.read.format('delta').option('versionAsOf',
0).load('/data/retail/sales_delta')

# MERGE INTO -- upsert new sales records (SQL API, e.g., in Databricks)
spark.sql('''
    MERGE INTO sales_delta AS target
    USING sales_updates AS source
    ON target.order_id = source.order_id
    WHEN MATCHED THEN UPDATE SET *
    WHEN NOT MATCHED THEN INSERT *
''')
```

## 5.5 CSV vs. Parquet vs. Delta — Choosing a Format



| Feature              | CSV                                                | Parquet                       | Delta Lake                                     |
|----------------------|----------------------------------------------------|-------------------------------|------------------------------------------------|
| Storage type         | Row-based, text                                    | Columnar, binary              | Columnar, binary (Parquet + log)               |
| Schema embedded      | No                                                 | Yes                           | Yes                                            |
| Compression          | None by default                                    | Yes (efficient)               | Yes (efficient)                                |
| ACID transactions    | No                                                 | No                            | Yes                                            |
| Time travel          | No                                                 | No                            | Yes                                            |
| Update / delete rows | No (rewrite whole file)                            | No (rewrite whole file)       | Yes (MERGE, UPDATE, DELETE)                    |
| Typical use case     | Data exchange, small files, ingestion landing zone | Analytics-ready batch storage | Lakehouse tables needing reliability & upserts |

## 5.6 Write Modes

Every writer call takes a mode that controls what happens if data already exists at the target path or table:

| Mode                    | Behavior                                                                      |
|-------------------------|-------------------------------------------------------------------------------|
| overwrite               | Replaces all existing data at the target with the new DataFrame's data.       |
| append                  | Adds the new data to whatever already exists, without touching existing rows. |
| errorIfExists (default) | Throws an error if data already exists at the target.                         |
| ignore                  | Silently does nothing if data already exists (no error, no write).            |

## 5.7 Partitioning on Write

Partitioning splits output data into subfolders based on the values of one or more columns (e.g., city=Ahmedabad/, city=Pune/). Queries that filter on the partition column can then skip reading entire folders, which is one of the single biggest performance levers available for large tables.

```
df.write.mode('overwrite').partitionBy('city', 'category').parquet('/data/output/sales_partitioned')

# A downstream query filtering on 'city' only reads the matching partition folder
spark.read.parquet('/data/output/sales_partitioned').filter("city = 'Pune'")
```

### Partitioning guidance

Choose partition columns with low-to-moderate cardinality that are frequently used in filters (e.g., city, order\_date). Avoid partitioning on high-cardinality columns like a unique order\_id, which creates too many tiny folders and hurts performance instead of helping it.

## Module 6: Hands-On Lab — Retail Sales Pipeline

This guided exercise ties together everything covered in the session. Participants should work in a Databricks notebook or local PySpark environment.

### Scenario

You have been given a raw CSV export of retail transactions from stores across Ahmedabad, Mumbai, Jaipur, and Pune. Your task is to clean it, analyze it with both the DataFrame API and SQL, and land it as an efficient, reliable Delta table.

### Step 1 — Load the Raw CSV

```
raw_df = (spark.read
    .option('header', True)
    .option('inferSchema', True)
    .csv('/data/retail/raw_sales.csv'))
raw_df.printSchema()
raw_df.show(5)
```

### Step 2 — Clean and Transform (DataFrame API)

```
from pyspark.sql import functions as F

clean_df = (raw_df
    .dropDuplicates(['order_id'])
    .na.drop(subset=['city', 'revenue'])
    .withColumn('revenue', F.col('revenue').cast('double'))
    .withColumn('order_month', F.date_format('order_date', 'yyyy-MM')))

clean_df.cache() # will be reused in the next steps
clean_df.count() # action -- triggers caching
```

### Step 3 — Analyze with Spark SQL

```
clean_df.createOrReplaceTempView('clean_sales')

spark.sql('''
    SELECT city, order_month, ROUND(SUM(revenue), 2) AS monthly_revenue
    FROM clean_sales
    GROUP BY city, order_month
    ORDER BY city, order_month
''').show()
```

### Step 4 — Write as a Partitioned Delta Table

```
(clean_df.write
```

```
.format('delta')
.mode('overwrite')
.partitionBy('city')
.save('/data/output/clean_sales_delta'))

spark.sql('''
CREATE TABLE IF NOT EXISTS clean_sales_delta
USING DELTA
LOCATION '/data/output/clean_sales_delta'
''')
```

## Step 5 — Simulate an Upsert

```
updates_df = spark.createDataFrame([
    (5001, 'Pune', '2026-06', 15250.0)
], ['order_id', 'city', 'order_month', 'revenue'])
updates_df.createOrReplaceTempView('sales_updates')

spark.sql('''
MERGE INTO clean_sales_delta AS target
USING sales_updates AS source
ON target.order_id = source.order_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
''')
```

## Discussion Questions for the Class

9. Why did we cache clean\_df before running multiple operations on it?
10. What would change if we had written Step 4 as CSV instead of Delta? What capability would we lose in Step 5?
11. Why is city a reasonable partition column here, but order\_id would not be?

# Module 7: Summary & Key Takeaways

## Key Concepts Recap

- **Spark architecture:** a Driver coordinates work; Executors run tasks in parallel across a cluster, managed by a Cluster Manager.
- **RDDs:** the low-level, immutable, distributed building block. Transformations are lazy; actions trigger execution via the DAG.
- **DataFrames:** structured, schema-aware, and optimized automatically by Catalyst — the recommended API for almost all workloads.
- **Spark SQL:** the same engine as DataFrames, accessible through familiar SQL syntax via temporary views.
- **CSV:** simple and universal, but slow and schema-less — best for ingestion, not analytics.
- **Parquet:** columnar, compressed, schema-embedded — the standard analytics-ready format.
- **Delta Lake:** Parquet plus a transaction log — adds ACID guarantees, time travel, schema enforcement, and MERGE support.

## Quick-Reference Cheat Sheet

| Task                   | Code                                                                   |
|------------------------|------------------------------------------------------------------------|
| Create SparkSession    | <code>spark = SparkSession.builder.appName('app').getOrCreate()</code> |
| Read CSV               | <code>spark.read.option('header', True).csv(path)</code>               |
| Read Parquet           | <code>spark.read.parquet(path)</code>                                  |
| Read Delta             | <code>spark.read.format('delta').load(path)</code>                     |
| Write (any format)     | <code>df.write.format(fmt).mode(mode).save(path)</code>                |
| Register temp view     | <code>df.createOrReplaceTempView('name')</code>                        |
| Run SQL                | <code>spark.sql('SELECT ...')</code>                                   |
| Cache a DataFrame      | <code>df.cache()</code>                                                |
| Inspect execution plan | <code>df.explain()</code>                                              |

## Suggested Further Practice

- Re-run the Module 6 lab using a real dataset (e.g., a public Kaggle retail dataset) instead of synthetic data.
- Compare `df.explain()` output between an RDD-based word count and the equivalent DataFrame `groupBy` to see Catalyst's optimizations directly.
- Explore Delta Lake's `DESCRIBE HISTORY` command to see the full audit trail of changes to a table.
- For Databricks learners: connect this session to Databricks-specific cluster and notebook concepts as a natural next module.